

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №2 по курсу
«Информационный поиск»**

Поисковый робот

Студент: К. А. Арусланов

Группа: М8О-403Б-23

Преподаватель: А. А. Кухтичев

Дата: 06.10.2026

Оценка:

Москва, 2026

Содержание

1	Постановка задачи	2
2	Метод решения	2
2.1	Источники и входные данные	2
2.2	Очередь и сохранение документов	2
2.3	Проверка изменений	2
2.4	Журнал выполнения и план тестирования	3
3	Результаты	3
4	Вывод	3

1 Постановка задачи

Для корпуса игровых публикаций требуется написать поисковый робот. Единственным аргументом при запуске должен быть путь к YAML-файлу с секциями `db` и `logic`. Робот должен сохранять в базе нормализованный URL, сырой HTML, название источника и время загрузки в формате Unix timestamp. После остановки обход должен продолжаться с незавершённого документа. Уже загруженные материалы требуется периодически проверять и заменять в базе только при изменении содержимого.

2 Метод решения

2.1 Источники и входные данные

Входом служит UTF-8 TSV-файл: короткий `urls.txt` из первой лабораторной или объединённый инвентарь `sitemap`. Последний содержит 1 055 674 URL-кандидата из восьми игровых сайтов. Файл читается построчно, импортируется в MongoDB партиями и не загружается целиком в память. URL нормализуется с удалением параметров отслеживания; его хост сверяется с разрешённым списком для соответствующего источника.

2.2 Очередь и сохранение документов

В коллекции `frontier` у каждого URL есть состояние, время следующей попытки, число попыток и служебные HTTP-заголовки. Для URL создаётся запись `documents` с полями `url`, `source`, `html`, `fetches_at` и `content_sha256`; также сохраняются результат парсера и HTTP-метаданные. Ключ `_id` равен нормализованному URL, поэтому повторная загрузка не создаёт дубликатов.

Робот атомарно выбирает доступный URL и отмечает его как обрабатываемый. Затем он проверяет `robots.txt` и делает запрос с паузой для каждого хоста. Редиректы проверяются по отдельности: переход за пределы источника не выполняется. После сохранения HTML URL помечается как завершённый. При перезапуске незавершённые URL возвращаются в начало очереди. Механизм рассчитан на один процесс робота.

2.3 Проверка изменений

При повторном запросе робот использует сохранённые ETag и Last-Modified. Ответ 304 оставляет HTML неизменным. Если сервер возвращает 200, вычисляется SHA-256 исходного ответа: при совпадении хеша документ также не перезаписывается. Для изменённого ответа обновляются HTML, разбор и `fetches_at`. Время каждой проверки отдельно хранится в `frontier.last_checked_at`. Ошибки и запрет `robots.txt` дают отложенную повторную попытку; для 429 учитывается Retry-After.

2.4 Журнал выполнения и план тестирования

При переносе компонентов из lab1 в lab2 добавлены парсеры GameMAG, GamingOnLinux, PC Gamer и Eurogamer. Для нового робота написаны отдельные тесты с имитацией MongoDB и HTTP-ответов. Проверяются:

- чтение YAML и оба формата входного TSV;
- загрузка и сохранение обязательных полей;
- повторный запуск после завершения и после остановки на URL;
- ответы 304, неизменённый 200 и изменённый 200;
- запрет robots.txt, отличие текстовых правил от HTML-заглушки и отложенная попытка после 429.

3 Результаты

Код робота запускается командой `python crawler.py crawler.example.yaml`. В примере ограничение `max_documents: 10` позволяет проверить небольшую часть списка `urls.txt`. Команда `python run_tests.py` проверяет систему без сервера MongoDB и без обращений к сайтам.

При локальном прогоне прошёл 21 тест, в том числе восемь тестов робота. Тестовые сценарии подтвердили сохранение HTML и Unix timestamp, немедленное возвращение прерванного URL в начало очереди, отсутствие перезаписи при 304 и совпадающем SHA-256, обновление документа при изменении HTML, соблюдение robots.txt и Retry-After. Исходные HTML-образцы первой лабораторной, доступные локально, дополнительно проверяются старыми тестами.

Также выполнен пробный запуск с MongoDB в Docker Compose. За несколько запусков из `urls.txt` сохранены все 24 документа восьми источников: 9,44 МБ HTML в кодировке UTF-8 и 200,556 символов извлечённого текста. Пустых текстов и ошибок парсера нет. Первые пять URL StopGame попали на повторную попытку из-за ложного обнаружения капчи в правиле `Clean-param` файла `robots.txt`. После исправления проверки все пять документов успешно сохранены; отложенных ошибок в очереди не осталось. Число URL из sitemap остаётся оценкой возможного объёма корпуса, а не числом уже загруженных документов. Инструкция по запуску приведена в `doc/lab2_run.md`. Исходный код размещён в ветке lab2 репозитория: <https://github.com/kirbasss/MAI-IR/tree/lab2>.

4 Вывод

В этой работе я дополнил сбор игровых публикаций устойчивой очередью в MongoDB. Теперь робот может продолжить обход после остановки и периодически проверять уже сохранённые страницы. Было полезно разобраться, как связать состояние очереди с сохранением документа и зачем при повторной проверке нужны ETag, Last-Modified и контрольная сумма ответа.

Я проверил робота тестами и пробной загрузкой в MongoDB. При массовой обкатке ещё нужно будет следить за ошибками сайтов и долей пригодных публикаций: sitemap даёт лишь предварительный список адресов.